
DRUG-TARGET BINDING AFFINITY PREDICTION

Complete Phase-by-Phase Reference: Code, Architecture & Reasoning (Phases 1–8)

Dual-GNN with Cross-Attention Fusion | PDBind Refined Set | CASF-2013 Benchmark

PHASE 1 — Data Ingestion & Verification

Goal: Before building anything, confirm every downloaded structure file (protein pocket + ligand) actually parses correctly, so broken files are caught now rather than mid-training.

Datasets Used

- **Refined set** (training/validation): PDBBind v2020 refined set, 5,318 complexes, sourced from Hugging Face (photonmz/pdbbindpp-2020)
- **Core set** (final test): CASF-2013 core set, 195 complexes, sourced via a Kaggle mirror
- Each complex folder contains files prefixed by PDB ID: <id>_protein.pdb, <id>_pocket.pdb, <id>_ligand.mol2, <id>_ligand.sdf

Imports

```
import os
import pandas as pd
from rdkit import Chem
from Bio.PDB import PDBParser
import warnings
warnings.filterwarnings("ignore")
```

os — filesystem navigation (listing folders, joining paths). **pandas** — builds the results table and writes the final CSV. **RDKit's Chem** — parses ligand files (.mol2/.sdf) and validates chemistry. **BioPython's PDBParser** — parses protein structure files (.pdb). The warnings filter silences non-critical library noise so only real issues are visible.

Core Function: verify_complex(folder_path)

```
pdb_id = os.path.basename(folder_path)
protein_path = os.path.join(folder_path, pdb_id + "_protein.pdb")
pocket_path = os.path.join(folder_path, pdb_id + "_pocket.pdb")
mol2_path = os.path.join(folder_path, pdb_id + "_ligand.mol2")
sdf_path = os.path.join(folder_path, pdb_id + "_ligand.sdf")
```

The PDB ID is extracted from the folder name itself, then used as a prefix to build every expected filename — this matches the actual on-disk naming convention discovered during setup (files are prefixed with their ID, not generic names like just "pocket.pdb").

```
if not os.path.exists(pocket_path):
    result["error"] = "missing " + pdb_id + "_pocket.pdb"
    return result

parser = PDBParser(QUIET=True)
structure = parser.get_structure(pdb_id, pocket_path)
residue_count = 0
for model in structure:
    for chain in model:
        for residue in chain:
            residue_count = residue_count + 1
if residue_count == 0:
    result["error"] = "pocket.pdb has zero residues"
    return result
```

First checks the pocket file exists at all. If present, BioPython parses it into a nested structure object (model → chain → residue) and every residue is counted by walking that hierarchy. A file that parses but contains zero residues is still marked invalid — existence alone isn't proof of usable data.

```
mol = None
if os.path.exists(mol2_path):
    mol = Chem.MolFromMol2File(mol2_path, sanitize=True)
if mol is None and os.path.exists(sdf_path):
    supplier = Chem.SDMolSupplier(sdf_path, sanitize=True)
    if len(supplier) > 0:
        mol = supplier[0]

if mol is None:
    result["error"] = pdb_id + "_ligand.mol2/.sdf missing or failed to parse"
    return result
if mol.GetNumAtoms() == 0:
    result["error"] = "ligand has zero atoms"
    return result

result["valid"] = True
return result
```

RDKit tries .mol2 first (preferred, since it explicitly encodes bond types). If that fails, it falls back to .sdf. `sanitize=True` forces RDKit to check chemical validity (valences, aromaticity) — not just that the text parses. A molecule with zero atoms is still rejected. Only after passing every check is `valid` set to `True`.

Batch Loop and Output

```
def run_verification(root_dir, source_label):
    folders = find_complex_folders(root_dir)
    rows = []
    for folder in folders:
        result = verify_complex(folder)
        result["source"] = source_label
        rows.append(result)
    return pd.DataFrame(rows)

combined = pd.concat([refined_results, core_results], ignore_index=True)
combined.to_csv(OUTPUT_CSV, index=False)
```

Loops `verify_complex()` across every folder in both datasets, tags each row with its source ("refined" or "core"), and writes everything — valid and invalid alike, with reasons for failure — to `verified_complexes.csv`. This file becomes the trusted source of truth for every later phase.

Actual Result

Refined set: 5,314 / 5,318 valid (99.92%) | Core set: 195 / 195 valid (100%) — only 4 complexes failed across the entire dataset.

PHASE 2 — Graph Construction

Goal: Convert every verified complex into two graph objects — one for the molecule, one for the protein pocket — each labeled with its true binding affinity (pKd), ready for the model.

The Missing Labels Problem

The Hugging Face refined set ships structure files only, with no affinity labels. This was resolved by pulling `LP_PDBBind.csv` from the LP-PDBBind GitHub repository (THGLab) — a research dataset providing PDB ID → pKd mappings for PDBBind 2020, filtered to just the "refined" category rows. The core set's own `pdbbind_v2013_core.csv` already had clean `pdb_id/label` columns.

Merging Labels with Verified Structures

```
refined_labels = pd.read_csv(LABELS_CSV, index_col=0)
refined_labels = refined_labels[refined_labels["category"] == "refined"]
refined_labels = refined_labels.reset_index()
refined_labels = refined_labels.rename(columns={"index": "pdb_id", "value": "label"})

core_labels = pd.read_csv(CORE_LABELS_CSV)
core_labels = core_labels[["pdb_id", "label"]]

all_labels = pd.concat([refined_labels, core_labels], ignore_index=True)
merged = verified.merge(all_labels, on="pdb_id", how="inner")
```

An inner merge means only complexes present in *both* the verified-structures list AND the labels file survive — anything missing a label, or missing a verified structure, is silently dropped rather than crashing later. Result: 5,339 usable complexes (5,146 refined + 193 core).

Building the Molecule Graph

```
def get_atom_features(atom):
    atomic_num = atom.GetAtomicNum()
    degree = atom.GetDegree()
    formal_charge = atom.GetFormalCharge()
    is_aromatic = 1 if atom.GetIsAromatic() == True else 0
    hybridization = int(atom.GetHybridization())
    return [atomic_num, degree, formal_charge, is_aromatic, hybridization]
```

Every atom becomes a node with a 5-number feature vector: which element it is, how many bonds it has, its electrical charge, whether it's part of an aromatic ring, and its hybridization state (sp/sp2/sp3). These are the raw signals the GNN learns from.

```
num_bonds = mol.GetNumBonds()
while bond_index < num_bonds:
    bond = mol.GetBondWithIdx(bond_index)
    start = bond.GetBeginAtomIdx()
    end = bond.GetEndAtomIdx()
    bond_type = float(bond.GetBondTypeAsDouble())
    edge_index_list.append([start, end])
    edge_attr_list.append([bond_type])
    edge_index_list.append([end, start])
    edge_attr_list.append([bond_type])
    bond_index = bond_index + 1
```

Every bond becomes an edge, added in *both* directions (start→end and end→start) since graph neural networks need bidirectional edges to let information flow both ways during message passing. Bond order (single/double/aromatic, as a float) is stored as the edge feature.

Building the Pocket Graph

```
def get_residue_ca_coord(residue):
    if "CA" in residue:
        return residue["CA"].get_coord()
    return None

def get_residue_features(residue):
    resname = residue.get_resname()
    one_hot = [0] * len(AMINO_ACIDS)
    if resname in AMINO_ACIDS:
        one_hot[AMINO_ACIDS.index(resname)] = 1
    return one_hot
```

Each residue's identity is encoded as a one-hot vector across the 20 standard amino acids. Its spatial position is taken from the CA (alpha-carbon) atom — the conventional "anchor point" for representing a residue's location in 3D space.

```
distance = (dx * dx + dy * dy + dz * dz) ** 0.5
if distance <= POCKET_DISTANCE_CUTOFF:
    edge_index_list.append([i, j])
```

Every residue pair is checked pairwise (an $O(n^2)$ loop); if two residues' CA atoms are within 8Å of each other, an edge is drawn between them. This defines the pocket's "contact network" — which parts of the protein are physically close enough to plausibly interact with a bound ligand.

Saving Combined Graphs

```
combined = {
    "pdb_id": pdb_id, "source": source, "label": float(label),
    "mol_x": mol_graph.x, "mol_edge_index": mol_graph.edge_index, "mol_edge_attr":
    mol_graph.edge_attr,
    "pocket_x": pocket_graph.x, "pocket_edge_index": pocket_graph.edge_index,
}
torch.save(combined, os.path.join(OUTPUT_DIR, pdb_id + ".pt"))
```

Each complex's molecule graph, pocket graph, and true pKd label are bundled into one dictionary and saved as a single .pt file — one file per complex, in the graphs/ folder. This is the exact format every later phase reads from.

Actual Result

5,339 / 5,339 complexes successfully converted to graphs. 0 failures.

PHASE 3 — Scaffold-Based Train/Val/Test Split

Goal: Split the refined-set complexes into train/validation groups without letting structurally near-identical molecules leak across the split — a well-known pitfall that inflates reported accuracy if done with a naive random split.

Computing a Murcko Scaffold

```
scaffold = MurckoScaffold.GetScaffoldForMol(mol)
scaffold_smiles = Chem.MolToSmiles(scaffold)
```

A **Bemis-Murcko scaffold** is a molecule's core ring framework with side chains stripped away. Two molecules differing only by a small substituent (e.g. a methyl vs. an ethyl group) share the same scaffold — meaning they are structural cousins that should never be split across train and test, since the model could effectively "see the answer" via a near-duplicate.

Grouping and Splitting

```
grouped = scaffold_df.groupby("scaffold")["pdb_id"].apply(list).reset_index()
grouped["group_size"] = grouped["pdb_id"].apply(len)
grouped = grouped.sort_values("group_size", ascending=False)

train_target = int(total_complexes * TRAIN_FRACTION)
for row in grouped.rows:
    if len(train_ids) < train_target:
        train_ids.extend(group_ids)
    else:
        val_ids.extend(group_ids)
```

Complexes are grouped by shared scaffold, and entire groups (not individual complexes) are assigned to train or val. Groups are processed largest-first, filling the train set to roughly 80% of the total — the remainder becomes validation. Because scaffold group sizes vary, the final split won't be an exact 80/20, which is expected.

Self-Verification

```
train_scaffolds = set(scaffold_df[scaffold_df["pdb_id"].isin(train_ids)]["scaffold"])
val_scaffolds = set(scaffold_df[scaffold_df["pdb_id"].isin(val_ids)]["scaffold"])
overlap = train_scaffolds.intersection(val_scaffolds)
print("Scaffold overlap between train and val:", len(overlap), "(should be 0)")
```

Before trusting the split, the script explicitly checks that no scaffold appears in both sets — not assumed, actually verified in the printed output.

Test Set Handling

The 193-complex CASF core set is never touched by this script — it's kept entirely separate and used only in Phase 6, matching how the field's published benchmarks work: train/validate freely, but the core set stays untouched until final evaluation.

Actual Result

Refined complexes split into train and validation groups by scaffold, zero scaffold overlap confirmed. Core set (193) reserved untouched as split_test.csv.

PHASE 4 — Model Architecture

Goal: Define the dual-GNN + cross-attention neural network that reads both graphs and outputs one predicted pKd number. No training happens in this phase — only architecture and a shape-check forward pass.

Molecule Encoder (GIN)

```
self.layers.append(GINConv(mlp)) # x3 layers

def forward(self, x, edge_index):
    x = self.input_proj(x)
    x = F.relu(x)
    for layer in self.layers:
        residual = x
        x = layer(x, edge_index)
        x = F.relu(x)
        x = self.dropout(x)
        x = x + residual # residual connection
```

Graph Isomorphism Network (GIN) layers were chosen for the molecule branch because they are specifically strong at distinguishing structurally different molecular graphs. Each layer lets an atom absorb information from its bonded neighbors ("message passing"); after 3 layers, each atom's representation reflects roughly its 3-bond-radius chemical neighborhood. Residual connections (adding the input back to the output) help gradients flow during training and stabilize deeper stacks.

Pocket Encoder (GAT)

```
self.layers.append(GATConv(hidden_dim, hidden_dim, heads=heads, concat=False, dropout=DROPOUT))
```

Graph Attention Network (GAT) layers were chosen for the pocket branch because attention weights are directly interpretable — useful for later visualizing which residues the model focuses on. 4 attention heads let each residue weigh its neighbors by learned importance rather than simple averaging. `concat=False` keeps the output dimension fixed at 128 regardless of head count.

Cross-Attention Fusion

```
self.mol_to_pocket_attn = nn.MultiheadAttention(hidden_dim, heads, batch_first=True)
self.pocket_to_mol_attn = nn.MultiheadAttention(hidden_dim, heads, batch_first=True)

def forward(self, mol_x, pocket_x):
    mol_attended, _ = self.mol_to_pocket_attn(mol_seq, pocket_seq, pocket_seq)
    pocket_attended, _ = self.pocket_to_mol_attn(pocket_seq, mol_seq, mol_seq)
    mol_fused = mol_x + mol_attended.squeeze(0)
    pocket_fused = pocket_x + pocket_attended.squeeze(0)
```

This is the component that makes the model reason about *compatibility* between the two structures rather than characteristics of each independently. Molecule atoms attend over pocket residues (learning which residues matter to which atoms) and vice versa. PyTorch's built-in `MultiheadAttention` performs the underlying scaled dot-product attention math.

Pooling and Regression Head

```

mol_pooled = mol_fused.mean(dim=0, keepdim=True)
pocket_pooled = pocket_fused.mean(dim=0, keepdim=True)
prediction = self.head(mol_pooled, pocket_pooled)

# RegressionHead.mlp:
nn.Linear(hidden_dim * 2, hidden_dim), nn.ReLU(), nn.Dropout(DROPOUT),
nn.Linear(hidden_dim, hidden_dim // 2), nn.ReLU(),
nn.Linear(hidden_dim // 2, 1)

```

Mean pooling collapses the variable number of atoms/residues into one fixed-size vector each (necessary since different molecules/pockets have different atom/residue counts). Both pooled vectors are concatenated and passed through a shrinking 3-layer MLP (256→128→64→1) ending in a single scalar: the predicted pKd.

Key Hyperparameters (Original / Iteration 1)

Setting	Value
HIDDEN_DIM	128
NUM_MOL_LAYERS / NUM_POCKET_LAYERS	3 / 3
ATTENTION_HEADS	4
DROPOUT	0.2

PHASE 5 — Training Loop

Goal: Train the model on the training split, monitor validation loss every epoch, save the best checkpoint, and stop automatically once the model stops improving.

The Core Training Step

```
for pdb_id in pdb_ids:
    data = load_graph(pdb_id)
    true_label = torch.tensor(data["label"], dtype=torch.float)

    optimizer.zero_grad()
    prediction = model(data["mol_x"], data["mol_edge_index"],
                       data["pocket_x"], data["pocket_edge_index"])
    loss = loss_fn(prediction, true_label)
    loss.backward()
    optimizer.step()
```

For every complex: load its two graphs, run a forward pass to get a prediction, compute **Mean Squared Error** against the true pKd, call `.backward()` to compute how much every weight contributed to the error (backpropagation), then `optimizer.step()` nudges every weight slightly toward reducing that error. In the original (Iteration 1) version, this update happens after every single complex — batch size of 1.

Validation Pass

```
model.eval()
with torch.no_grad():
    prediction = model(...)
    loss = loss_fn(prediction, true_label)
```

Same forward pass, but `model.eval()` disables dropout, and `torch.no_grad()` skips gradient computation entirely — this is pure measurement, no learning happens here. Comparing train loss vs. val loss each epoch is how overfitting gets detected in real time.

Checkpointing and Early Stopping

```
if val_loss < best_val_loss:
    best_val_loss = val_loss
    epochs_without_improvement = 0
    torch.save(model.state_dict(), CHECKPOINT_PATH)
else:
    epochs_without_improvement = epochs_without_improvement + 1

if epochs_without_improvement >= PATIENCE: # PATIENCE = 10
    break
```

Every time validation loss hits a new low, that exact model state is saved — so the final saved model is the *best* one seen, not necessarily the last epoch trained. If 10 consecutive epochs pass with no improvement, training stops automatically, even before the 100-epoch ceiling — preventing wasted compute on a model that has plateaued or started overfitting.

Actual Result — Iteration 1 (Original Settings)

40 epochs run, early stopping triggered. Best validation MSE: 2.6232 (epoch 30). Train MSE at that point: ~2.38 — a gap that widened in later epochs, an early sign of overfitting confirmed later in Phase 6.

Regularization Attempts (Iterations 2 & 3)

After Phase 6 revealed the model underperformed a simple baseline, two follow-up iterations were tried:

- **Iteration 2** (HIDDEN_DIM=64, dropout=0.3, weight_decay=1e-4, batch=16, LayerNorm added): resulted in *underfitting* — val loss plateaued around 3.27, both train and val stuck at a mediocre error.
- **Iteration 3** (HIDDEN_DIM=96, dropout=0.25, weight_decay=5e-5, batch=8): began *overfitting again*, faster than the original.
- **Conclusion:** the original Iteration 1 settings were restored and retrained (saved separately as `best_model_final.pt`) as the best result obtained despite showing overfitting — documented honestly as an open limitation rather than papered over.

PHASE 6 — Evaluation on the Held-Out Test Set

Goal: Measure real performance on the 193 CASF core-set complexes — data the model never saw during training or validation — and compare against a simple baseline to check whether the architecture's complexity was justified.

Loading the Trained Model

```
model = BindingAffinityModel()
model.load_state_dict(torch.load(CHECKPOINT_PATH))
model.eval()
```

Loads the saved weights into the architecture and switches to evaluation mode (disables dropout) for consistent, deterministic predictions.

Computing RMSE and Pearson Correlation

```
def compute_rmse(true_values, predicted_values):
    mse = mean_squared_error(true_values, predicted_values)
    return mse ** 0.5

corr, _ = pearsonr(true_labels, predictions)
```

RMSE reports the average error in the same units as pKd itself — directly interpretable (e.g. RMSE 2.43 means predictions are typically off by about 2.43 pKd units). **Pearson correlation** measures whether the model's *ranking* of complexes (which binds stronger vs. weaker) matches the true ranking, independent of absolute accuracy.

The Baseline Comparison

```
fingerprint = AllChem.GetMorganFingerprintAsBitVect(mol, radius=2, nBits=1024)
rf = RandomForestRegressor(n_estimators=200, random_state=42, n_jobs=-1)
rf.fit(train_X, train_y)
```

A Random Forest trained purely on Morgan/ECFP molecular fingerprints (no protein structure information at all) serves as a deliberately simple point of comparison. If the far more complex dual-GNN can't beat this, that's a genuine, honest signal that the added architectural complexity isn't earning its keep on this dataset.

Actual Results

Model	Test RMSE	Pearson r
Dual-GNN (Iteration 1, final)	2.297 / 2.434 (two runs)	0.343 / 0.268
Random Forest baseline (fingerprints only)	1.811	0.598

Honest conclusion: the GNN did not beat the simple baseline. Training curves showed overfitting (validation loss plateaued while training loss kept falling); multiple regularization attempts either underfit or re-overfit. This is documented as a real, open finding — not hidden — and is a legitimate outcome in ML research: correctly diagnosing why a sophisticated model underperforms a simple one is itself a demonstrable skill.

PHASE 7 — User Interface (Streamlit App)

Goal: Build a usable interface wrapping the trained model — not changing model performance, only making results accessible and presentable.

Model Loading (Cached)

```
@st.cache_resource(show_spinner=False)
def load_model():
    model = BindingAffinityModel()
    model.load_state_dict(torch.load(CHECKPOINT_PATH, map_location="cpu"))
    model.eval()
    return model
```

`@st.cache_resource` ensures the model loads into memory only once per session, not on every single interaction — important since Streamlit re-runs the whole script on every button click.

Tab 1 — Single Prediction

```
selected_id = st.selectbox("Select a protein-ligand complex...", test_ids)
if st.button("Predict"):
    data = torch.load(os.path.join(GRAPHS_DIR, selected_id + ".pt"))
    prediction = predict(model, data["mol_x"], data["mol_edge_index"],
                        data["pocket_x"], data["pocket_edge_index"])
```

Lets the user pick any of the 193 held-out test complexes and see the model's prediction directly against the true, verified pKd value — a fixed, real pairing loaded straight from the Phase 2 graph files. Used for spot-checking accuracy on known answers.

Tab 2 — Batch Virtual Screening

```
pocket_file = st.file_uploader("Protein pocket file (.pdb)")
ligand_files = st.file_uploader("Candidate ligand files", accept_multiple_files=True)

pocket_x, pocket_edge_index = build_pocket_graph_from_bytes(pocket_file.read())
for ligand_file in ligand_files:
    mol_x, mol_edge_index = build_molecule_graph_from_bytes(...)
    prediction = predict(model, mol_x, mol_edge_index, pocket_x, pocket_edge_index)
```

This is the real-world screening workflow: one protein pocket uploaded once, then *any number* of candidate ligands scored against it in a single run — unlike Tab 1, these pairings are arbitrary and don't need to exist in the training data. Results are ranked highest predicted pKd first and offered as a downloadable CSV.

Toxicity Structural Alerts (PAINS + BRENK)

```
_filter_params.AddCatalog(FilterCatalogParams.FilterCatalogs.PAINS)
_filter_params.AddCatalog(FilterCatalogParams.FilterCatalogs.BRENK)
TOXICITY_CATALOG = FilterCatalog.FilterCatalog(_filter_params)

def check_toxicity_alerts(mol):
    if TOXICITY_CATALOG.HasMatch(mol):
        return "Warning: " + TOXICITY_CATALOG.GetFirstMatch(mol).GetDescription()
    return "No alerts"
```

A deliberately simple, deterministic addition (Phase 8's scope, integrated directly into the app): RDKit ships built-in catalogs of known problematic chemical substructures (PAINS — assay-interference patterns; BRENK — reactive/unstable groups), curated from real medicinal chemistry literature. This is a rule-based pattern match, explicitly *not* a trained ML toxicity model — every candidate ligand is checked against both catalogs and flagged if a match is found.

Bug Fixed: HTML Rendering

An early version of the "Recent Predictions" card rendered as raw HTML text instead of a styled card. Cause: Streamlit's Markdown parser treats any line indented 4+ spaces as a code block, overriding HTML rendering. Fix: built the HTML string via plain + concatenation instead of an indented triple-quoted f-string, removing the accidental leading whitespace.

PHASE 8 — Toxicity Screening Addition & Project Wrap-Up

Goal: Add a lightweight, honest toxicity-risk signal to the batch screening results, and finalize the project for sharing (GitHub, documentation).

Why Not a Trained Toxicity Model

The original inspiration (a separate reference project, "ReBind") used a fine-tuned ChemBERTa transformer trained on four pharma datasets (ClinTox, Tox21, SIDER, BBBP). Fully replicating that would mean training four new models — a multi-week separate project, not a scoped addition. Given the timeline, the simplest legitimate alternative was chosen instead: RDKit's built-in structural alert catalogs, which require zero training and zero new dependencies (RDKit was already installed).

Honest Framing

The UI explicitly labels this feature as a rule-based pattern match, not a probability estimate from a trained model — matching the same honesty standard applied to the binding-affinity model's own overfitting disclosure earlier in the project.

Version Control & Publishing

```
git init
git add .
git commit -m "Initial commit: dual-GNN binding affinity predictor"
git remote add origin https://github.com/<user>/drug-binding-affinity-prediction.git
git branch -M main
git push -u origin main
```

A `.gitignore` excludes the multi-gigabyte `data/` and `graphs/` folders (GitHub isn't built for datasets this large, and has a 100MB per-file limit). Only code, configuration, and small result CSVs are versioned.

README

A top-level `README.md` was written summarizing: the problem statement, architecture, dataset sources with direct links, the actual results table (reported honestly, including the baseline comparison), repository structure, and setup/run instructions — the first thing any visitor to the repository sees.

Final Project Summary

Phase	Deliverable
1	verified_complexes.csv — 5,509 structurally valid complexes
2	graphs/ — 5,339 complete (graph + label) training examples
3	split_train.csv / split_val.csv / split_test.csv — leak-proof split
4	phase4_model.py — dual-GNN + cross-attention architecture
5	best_model_final.pt — trained checkpoint (val MSE 2.62—2.81)
6	phase6_predictions.csv — honest test RMSE 2.30—2.43 vs. baseline 1.81

7	app.py — Streamlit UI, single + batch prediction modes
8	Toxicity alerts, GitHub repo, README

Overall takeaway for presenting this project: a complete, real, working pipeline was built end-to-end — data verification through a usable UI — with every dataset, architectural choice, and result reported honestly, including the model's underperformance against a simple baseline. The diagnostic process (three training iterations, each with clear reasoning) is itself a demonstrable skill, not a weakness to hide.